UT5
Bases de Datos NoSQL

ACCESO A DATOS

2º DAM

Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial 4.0 Internacional.

Contenidos

- NoSQL Concepto
  - Diferencias Modelo Relacional
  - Ventajas
  - Inconvenientes
  - Tipos BD: documentales, clave-valor, columnas, OO, grafo
- MongoDB
  - ¿Cuándo utilizarla?
  - ¿Cuándo no utilizarla?
  - Instalación
  - Conceptos
  - Colecciones
  - Insertar
  - Consultas
  - Modificar
  - Borrar

Contenidos

- Neo4J
  - ¿Cuándo utilizarla?
  - ¿Cuándo no utilizarla?
  - Instalación
  - Conceptos
  - Grafo
    - Nodos, Etiquetas, Relaciones
  - Lenguaje CQL (Cypher Query Language)
    - Insertar
    - Consultas
    - Modificar
    - Borrar

NoSQL Concepto

- Las aplicaciones web generan información de forma masiva
  - Millones de usuarios accediendo simultáneamente y generando consultas a BD
- Las BD relaciones presentan problemas de escalabilidad y rendimiento
- La estructura E-R no se ajusta a la estructura de la información generada en la web
  - Objetos, XML, Grafos …
- Se requieren nuevos sistemas de almacenamiento de la información para solucionar las nuevas necesidades de la web
  - NoSQL al rescate – se conoce como “Big Data”

NoSQL Diferencias Relacional

- No se usa SQL como lenguaje de consultas
  - Cassandra utiliza el lenguaje CQL, MongoDB utiliza JSON o BigTable hace uso de GQL
- No se utiliza la estructura de tabla para el almacenamiento
  - Se basan en objetos, grafos, pares clave-valor …
- No se permiten operaciones JOIN
  - Para evitar la sobrecarga que generaría un join con los volúmenes de datos generados en la web
- Arquitectura distribuida
  - La información se distribuye entre varias máquinas para mejorar el rendimiento

NoSQL Ventajas

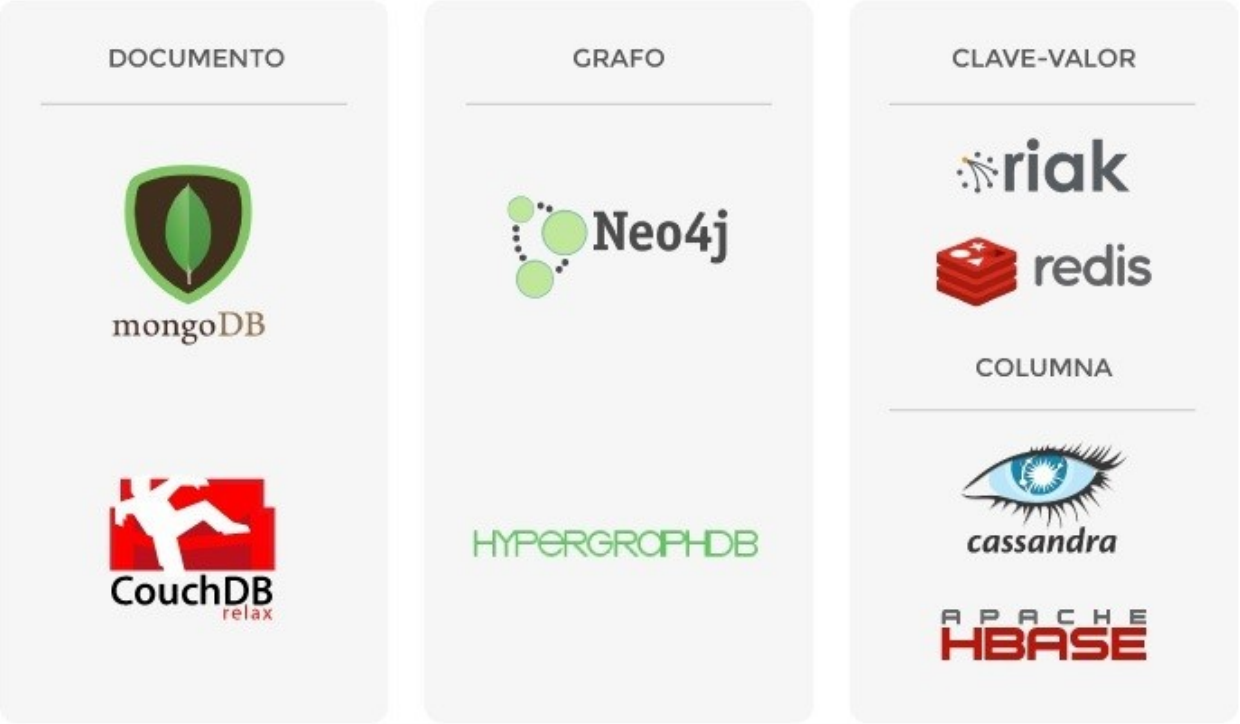
- Máquinas con pocos recursos
  - Requieren menos computación que un SGBD Relacional por lo que se requieren menos recursos HW
- Escalabilidad horizontal
  - Al ser un sistema distibuido el rendimiento se puede mejorar añadiendo más nodos
- Información escalable
  - Al ser distribuidos son capaces de manejar grandes volúmenes de datos
- Se evita el principal cuello de botella de un SGBD basado en SQL
  - Para ejecutar cada sentencia hay que transcribirla, si se producen muchas peticiones se ralentiza el sistema

NoSQL Inconvenientes

- “No son la panacea”
- Dependiendo de los requisitos del sitio pueden ser más eficientes pero para muchos sitios un sistemas de bases de datos relacional sigue siendo lo óptimo.
- Sólo se deben utilizar cuando son necesarias y van a mejorar realmente el rendimiento.
  - El modelo de datos cambia rápidamente y se necesita un alto rendimiento y escalabilidad, es probable que lo que mejor sean soluciones NoSQL.
  - Si por el contrario, la estructura de los datos no suele cambiar con normalidad y el crecimiento de los datos es “moderado”, las tecnologías SQL serán la mejor opción

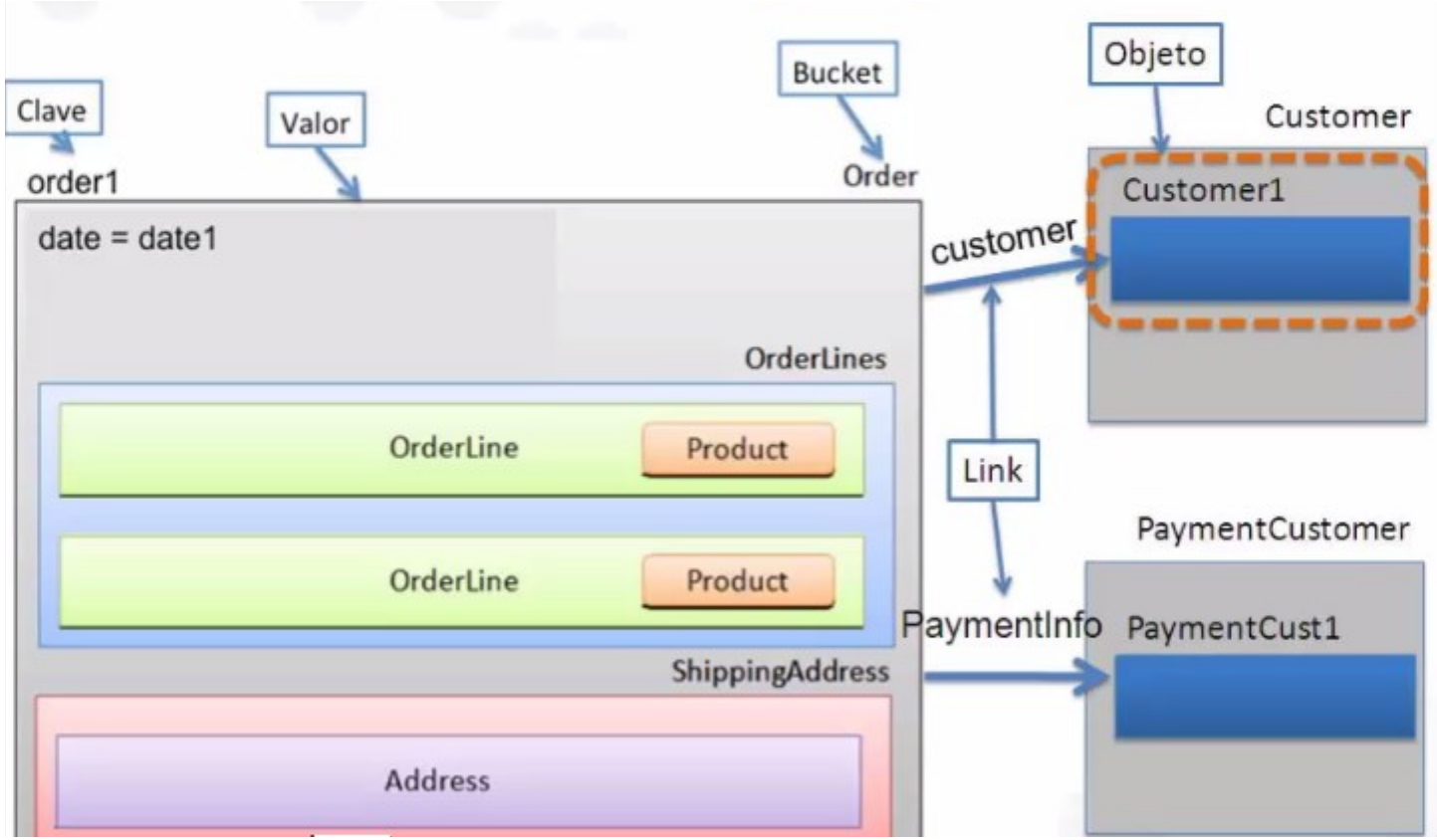
NoSQL – Tipos de BD

- Clave – Valor
- Columnas
- Documentales
- Grafo
- Orientadas a Objetos



NoSQL – Tipos de BD

- Clave – Valor
  - Son las más sencillas de entender. Guardan tuplas que contienen una clave y su valor. Cuándo se quiere recuperar un dato, se busca por su clave y se recupera el valor. El valor se suele almacenar como un binario (BLOB), lo que hace muy eficientes las lecturas y escrituras.



NoSQL – Tipos de BD

- Clave – Valor
  - Ejemplos:
    - DynamoDB: desarrollada por Amazon, es una opción de almacenaje que puedemos usar desde los Amazon Web Services. La utilizan el Washington Post y Scopely.
    - Redis: desarrollada en C y de código abierto, es utilizada por Craiglist y Stack Overflow (a modo de caché).

NoSQL – Tipos de BD

- Columnas
  - Pensadas para realizar consultas y agregaciones sobre grandes cantidades de datos. Funcionan de forma parecida a las bases de datos relacionales, pero almacenando columnas de datos en lugar de registros. Todos los casos de un solo elemento de datos (por ejemplo, Nombre de Persona) se almacenan de modo que se puede acceder como una unidad

    De gran utilidad en consultas analíticas (Data mining o Bussiness Inteligence)

Id	Apellido	Nombre	Salario
1	Gómez	José	40000
2	Jiménez	Mary	50000
3	Salabarrieta	Cathy	44000

El sistema por filas lo serializa:

```
1, Gómez, Jose, 40000;
2,Jimenez,Mary,50000;
3,Gutierrez,Cathy,44000;
```

Pero el sistema por COLUMNAS lo serializa así:

```
1,2,3;
Gómez,Jiménez, Gutiérrez;
José, Mary, Cathy;
40000,50000,44000;
```

NoSQL – Tipos de BD

- Columnas
  - Ejemplos:
    - Cassandra: incluida en esta sección, aunque en realidad sigue un modelo híbrido entre orientada a columnas y clave-valor. Es utilizada por Facebook y Twitter (aunque dejaron de usarla para almacenar tweets).
    - HBase. Escrita en Java y mantenida por el Projecto Hadoop de Apache, se utiliza para procesar grandes cantidades de datos. La utilizan Facebook, Twitter o Yahoo.

NoSQL – Tipos de BD

- **Documentales**
  - Son aquellas que gestionan datos semi estructurados. Es decir documentos. Estos datos son almacenados en algún formato estándar como puede ser XML ó JSON. Permiten consultas sobre el contenido de los documentos. Son las bases de datos NoSQL más versátiles. Se pueden utilizar en gran cantidad de proyectos, incluyendo muchos que tradicionalmente funcionarían sobre bases de datos relacionales

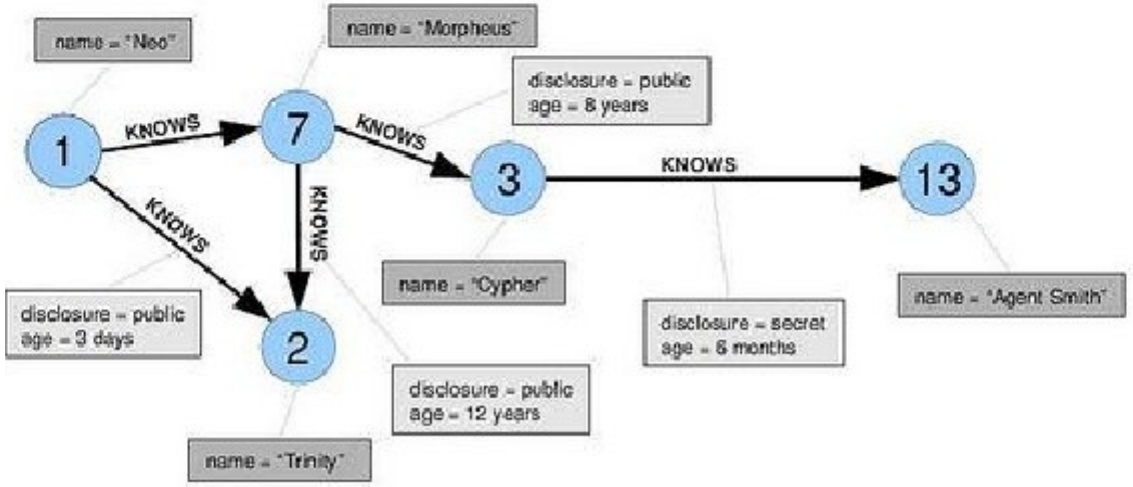
```
{
  Name: "Genbeta Dev",
  Tipo: "Blogging",
  Categorias:
  [
    {
      Título: "Desarrollo",
      Articulos: 89
    },
    {
      Título: "Formación",
      Artículos: 45
    }
  ]
}
```

NoSQL – Tipos de BD

- Documentales
  - Ejemplos:
  - MongoDB: probablemente la base de datos NoSQL más famosa del momento. En octubre del 2013, MongoDB conseguía 150 millones de dólares en financiación, convirtiéndose en una da las startups más prometedoras. Algunas compañías que actualmente utilizan MongoDB son Foursquare o eBay.
  - CouchDB: es la base de datos orientada a documentos de Apache. Este sistema es utilizado por compañías como Credit Suisse y la BBC

NoSQL – Tipos de BD

- **Grafo**
  - Basadas en la teoría de grafos utilizan nodos y aristas para representar los datos almacenados. Son muy útiles para guardar información en modelos con muchas relaciones, como redes sociales. Para que sea eficiente se debe normalizar de forma que cada nodo tenga una sola columna y cada relación dos.



NoSQL – Tipos de BD

- Grafo
  - Ejemplos:
    - Infinite Graph: escrita en Java y C++ por la compañía Objectivity. Tiene dos modelos de licenciamiento: uno gratuito y otro de pago.
    - Neo4j: base de datos de código abierto, escrita en Java por la compañía Neo Technology. Utilizada por compañías como HP, Infojobs o Cisco.

NoSQL – Tipos de BD

- **Orientadas a Objetos**
  - La información se almacena como objetos igual que en un lenguaje de programación orientado a objetos como Java.
  - Técnicamente podrían considerarse bases de datos NoSQL aunque no encajan en el concepto NoSQL de no relacionales, distribuidas y escalables.
  - Ejemplos:
    - Db4o: adquirida en 2008 por Versant, actualmente tienen su propio producto y y desde 2011 ya no trabajan en ella. Se puede descargar la versión libre.
    - Zope: es un framwork con servidor de aplicaciones para el desarrollo de sitios web OO con python

MongoDB

- **MongoDB** MongoDB es un sistema de base de datos **multiplataforma** orientado a **documentos**, de esquema libre.
- **¿Cuándo utilizarlo?**
  - Cualquier aplicación que necesite almacenar datos semi estructurados puede usar **MongoDB**. Es el caso de las típicas aplicaciones CRUD o de muchos de los desarrollos web actuales
  - Es especialmente útil en entornos que requieran escalabilidad. Con sus opciones de replicación y sharding, podemos conseguir un sistema que escale horizontalmente sin demasiados problemas

MongoDB

- **¿Cuándo NO utilizarlo?**
- En esta base de datos no existen las transacciones. Aunque nuestra aplicación puede utilizar alguna técnica para simular las transacciones, MongoDB no tiene esta capacidad. Solo garantiza operaciones atómicas a nivel de documento. Si las transacciones son algo indispensable en nuestro desarrollo, deberemos pensar en otro sistema.
- Tampoco existen los JOINS. Para consultar datos relacionados en dos o más colecciones, tenemos que hacer más de una consulta. En general, si nuestros datos pueden ser estructurados en tablas, y necesitamos las relaciones, es mejor que optemos por un RDBMS clásico

MongoDB

- **Instalación**
  - Seguiremos las instrucciones de la página dependiendo del SO y la versión que queramos.
  - En nuestro caso, versión 4.2 de MongoDB en un Ubuntu 18.04 (bionic): https://docs.mongodb.com/manual/tutorial/install-mongodb-on-ubuntu

MongoDB

- **Post-Instalación**
  - Arranque:

    ```
    sudo systemctl start mongod
    ```
  - Configurar MongoDB como un servicio para no tener que levantarlo a mano manualmente

    ```
    sudo systemctl enable mongod
    ```
  - Parada:

    ```
    sudo systemctl stop mongod
    ```

MongoDB

- **Shell de Mongo**
  - Para acceder a la shell, desde Terminal escribimos:
    `mongo`
  - Desde el prompt podemos ver en qué base de datos estamos con:
    `db`
  - Para ir a otra base de datos:
    `use <database>`
  - Para mostrar las bases de datos disponibles:
    `show dbs`

Actividad

- **Realizar Ejercicio 1 de Ejercicios1.pdf**

MongoDB

- **Conceptos**
- Un **documento** es una estructura de datos al estilo JSON (pares campo-valor). Estos documentos son almacenados en BSON, que es una representación binaria de JSON.
- El valor de cada campo puede ser de cualquier tipo **BSON**, incluido otro documento, arrays y arrays de documentos

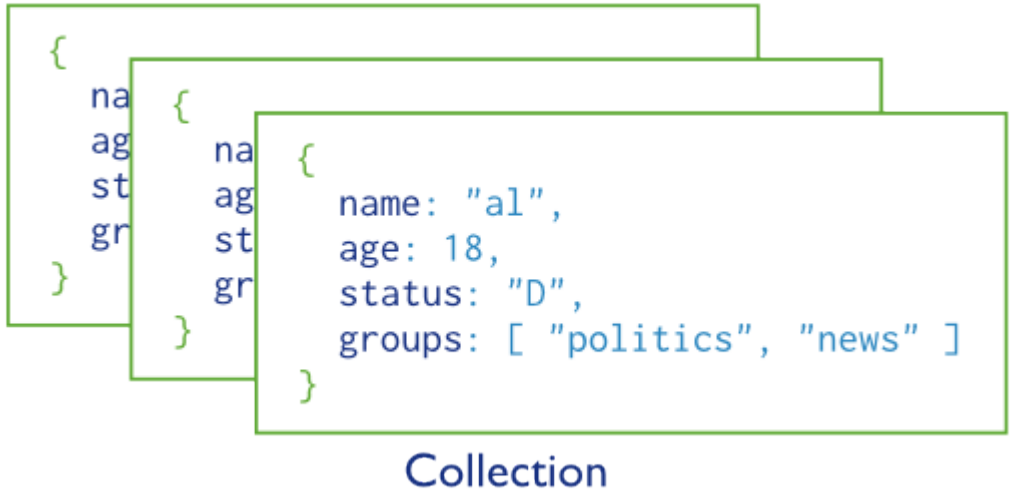
```
{
   field1: value1,
   field2: value2,
   field3: value3,
   ...
   fieldN: valueN
}
```

```
var mydoc = {
                _id: ObjectId("5099803df3f4948bd2f98391"),
                name: { first: "Alan", last: "Turing" },
                birth: new Date('Jun 23, 1912'),
                death: new Date('Jun 07, 1954'),
                contribs: [ "Turing machine", "Turing test", "Turingery" ],
                views : NumberLong(1250000)
            }
```

MongoDB

- Todos los documentos se almacenan en **colecciones.** Una colección es un grupo de documentos relacionados que comparten un conjunto de índices común.

- Una de las diferencias más importantes con respecto a las bases de datos relacionales, es que **no es necesario seguir un esquema**. Los documentos de una misma colección – concepto similar a una tabla de una base de datos relacional -, pueden tener esquemas diferentes



MongoDB

- Imaginemos que tenemos una colección a la que llamamos Personas. Un documento podría almacenarse de la siguiente manera

```
{
  Nombre: "Pedro",
  Apellidos: "Martínez Campo",
  Edad: 22,
  Aficiones: ["fútbol","tenis","ciclismo"],
  Amigos: [
   {
     Nombre:"María",
     Edad:22
   },
   {
     Nombre:"Luis",
     Edad:28
   }
  ]
}
```

- En la misma colección podríamos guardar un documento como este

```
{
  Nombre: "Luis",
  Estudios: "Administración y Dirección de Empresas",
  Amigos:12
}
```

- Este documento no sigue el mismo esquema que el primero. Tiene menos campos, algún campo nuevo que no existe en el documento anterior e incluso un campo de distinto tipo.
- Esto que es algo impensable en una base de datos relacional, es algo totalmente válido en MongoDB

MongoDB - Bases de Datos

- Por defecto se crea una Base de datos llamada test que podemos referenciar en el cliente con **db**
- Para crear una BD nueva
  - `use <nombre>`
  - Crea la BD y pasa a ser la BD actual en uso
  - Ahora db referencia nuestra nueva BD

MongoDB - Colecciones

- Una vez que ya tenemos nuestra BD podemos crear colecciones (Equivalente a tablas en Relacional)
- Para crear una colección en el cliente de MongoDB escribimos:

```
db.createCollection(<name>, { capped: <boolean>,
                              autoIndexId: <boolean>,
                              size: <number>,
                              max: <number>,
                              storageEngine: <document> } )
```

  - `db.createCollection("personas")`

Actividad

- Abre el cliente de MongoDB
  - Crea una base de datos nueva llamada `poblacion`
  - Teclea el comando db para comprobar que se ha creado correctamente y que es la BD en uso
  - En la BD población crea una colección llamada personas
  - Teclea el comando `show collections` para comprobar que la colección se ha creado correctamente

MongoDB - Insertar

- Una vez que ya tenemos una colección ya podemos insertar documentos en ella
  - Si la colección no existe la crea
  - Nos devuelve un documento WriteResult que

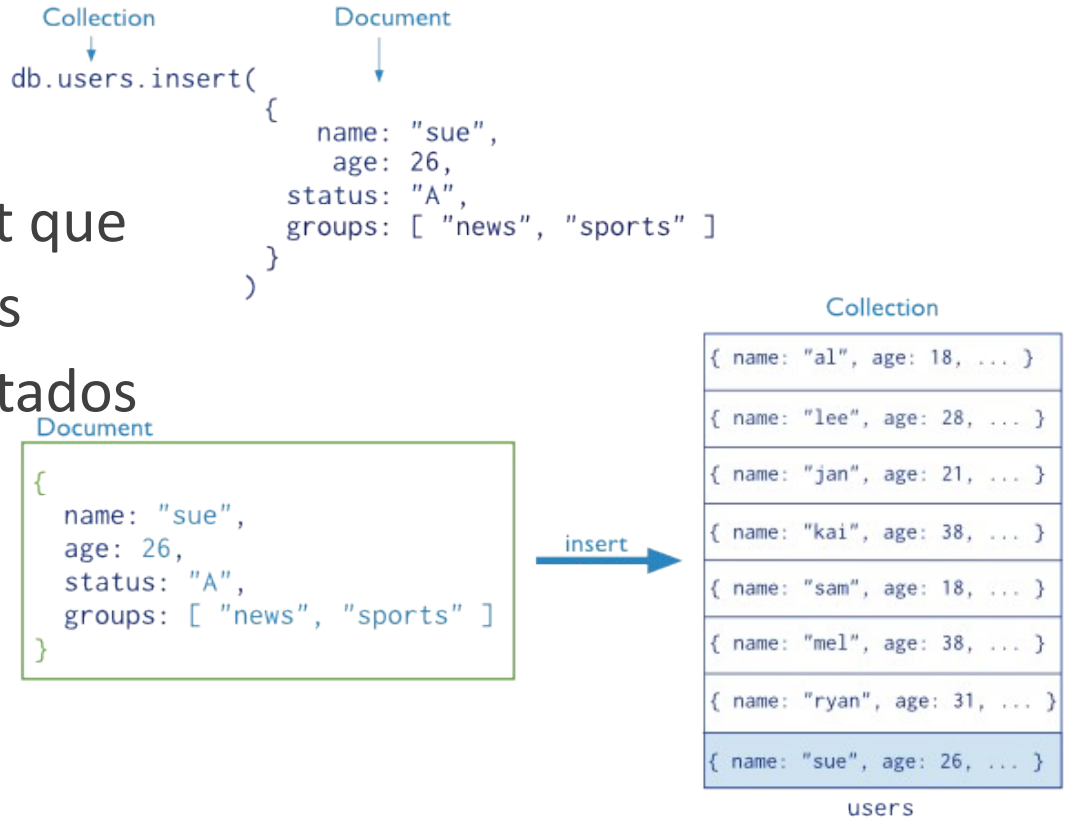
Indica el estado, si se ejecuta con éxito nos

Devuelve el número de documentos insertados

sino nos devolverá el error.

  - Podemos consultar los documentos

Insertados con **db.coleccion.find()**



Actividad

Abre el cliente de MongoDB

- En la colección llamada Personas inserta los documentos de la transparencia 26.
- Además inserta este documento:

```
{ Nombre: "Juan", Apellidos: "García Campo", Edad: 19,
 Aficiones: ["correr", "saltar","ciclismo"],Amigos: [
        { Nombre:"Pepa", Edad:27},
        { Nombre:"Luis", Edad:28} ]
 }
```

Después consulta los documentos insertados con

```
db.personas.find()
```

MongoDB – Consultas

- Seleccionar todos los documentos de una colección
  - db.coleccion.find()
- Seleccionar por un campo con un valor
  - db.coleccion.find().{ <field>: <value> }
  - Si el campo es un documento anidado utilizo el operador punto <field.field>
- Seleccionar especificando condiciones con operadores
  - db.coleccion.find().{ <field>: {operador:<values>} }}
  - Los operadores que se pueden usar son:
    - $eq, $gt, $gte, $lt, $lte, $ne, $in, $nin
    - http://docs.mongodb.org/manual/reference/operator/query/#query-selectors

MongoDB – Consultas

- Seleccionar con operadores lógicos
  - **AND**
  - Basta con especificar todos los valores de campos que queramos
  - Por ejemplo: db.coleccion.find({ type: 'food', price: { $lt: 9.95 } })
  - También con el operador $and
    - db.coleccion.find { $or: [{ <expression1> }, { <expression2> }, ... , { <expressionN> }] }

  - **OR**
  - db.coleccion.find { $or: [{ <expression1> }, { <expression2> }, ... , { <expressionN> }] }

  - **NOT**
  - db.coleccion.find{ field: { $not: { <operator-expression> } } }

Actividad

- Ejecuta las siguientes consultas en la colección personas. Asegúrate de entender lo que está haciendo cada una
  - `db.personas.find()`
  - `db.personas.find( { Nombre: { $in: ['Luis', 'Pedro' ] } } )`
  - `db.personas.find( { Nombre: 'Pedro', Edad: { $lt:22  } } )`
  - `db.personas.find( { Nombre: 'Pedro', Edad: { $lte:22  } } )`
  - `db.personas.find( { 'Amigos.Nombre': 'Luis' } )`
  - `db.personas.findOne()`

Actividad

- Ejecuta las siguientes consultas en la colección personas. Aségurate de entender lo que está haciendo cada una
  - db.personas.find(
    {
      $or: [{ Edad: { $gt: 20} }, { Amigos: { $lt: 15} }]
    }
)
  - db.personas.find(
    {
      Nombre: 'Luis',
      $or: [{ Edad: { $gt: 20} }, { Amigos: { $lt: 15} }]
    }
)

MongoDB – Consultas

- Consultas sobre Arrays
  - Valor exacto: especifico todos los valores del Array
    - db.personas.find({ Aficiones: ["correr", "saltar", "ciclismo"] })
  - Un elemento de un array: que al menos uno de sus elementos sea este
    - db.personas.find({ Aficiones: "ciclismo"})
  - Un elemento de una posición del array: que el valor del elemento de esta posición sea el que se indica (Posición indicada por el operador punto)
    - db.personas.find({'Aficiones.2': "ciclismo"})
  - Múltiples criterios operador $elemMatch: al menos un elemento cumpliendo todos los criterios
    - db.personas.find({ Amigos: { $elemMatch: { Nombre: 'María', Edad:{$lt: 28} } })

Actividad

- Prueba los ejemplos de las consultas sobre Arrays de la transparencia anterior.
- Asegúrate de entender los resultados que devuelven.

MongoDB – Consultas

- Limitar campos devueltos en las consultas
  - Por defecto el campo _id siempre se devuelve
  - Para especificar los campos que queremos que devuelva una consulta se usa la sintaxis
    - <field>: <1 or true> si queremos que se incluya
    - <field>: <0 or false> si no queremos que se incluya
  - Por ejemplo:
    - db.personas.find({ Nombre: { $in: ['Luis', 'Pedro'] }} , {Nombre:1, Aficiones:1})
      - Sólo devuelve los campos a 1 y el id
    - db.personas.find({ Nombre: { $in: ['Luis', 'Juan'] } }, {Nombre:1, Aficiones:1,_id:0})
      - Excluimos el id que se incluye por defecto
    - db.personas.find({ Nombre: { $in: ['Luis', 'Juan', 'Pedro'] }} , {Aficiones:0,Amigos:0})
      - Devuelve todos los campos salvo los excluidos

Actividad

- Prueba los ejemplos de las consultas de la transparencia anterior.
- Asegúrate de entender los resultados que devuelven.

MongoDB – Modificar

- Método update() para modificaciones (Actualiza un único documento)
  - Operadores de update para actualizar valores de campos
    http://docs.mongodb.org/manual/reference/operator/update/
    - $set, $inc,$mul, $rename, $min, $max, $currentDate …

```
db.personas.update(
   { Nombre: "Pedro" },
   {
     $set: {
       Apellidos: "cambiado",
       Aficiones: ["triatlon", "Nadar" ]
     },
     $min: { Edad: 15} }
)
```

MongoDB – Modificar

- Modificar campos de documentos anidados
  - Utilizamos el operador punto
    - db.personas.insert({ Nombre: "Lola", Apellidos: "García Campo", Edad: 19, Direccion:{ Calle:"Mi calle", Numero:5} })
    - db.personas.update({ Nombre: "Lola" },{ $set: {"Direccion.Numero": 7 } })
- Actualizar múltiples documentos con la opción multi
  - db.personas.update({ Nombre: { $in: ['Lola', 'Pedro'] } },{ $set: {Nombre: "Luis" } })
  - db.personas.update(

```
{ Nombre: "Luis" },
{
  $set: {
    Apellidos: "cambiado",
    Aficiones: ["triatlon", "Nadar" ]
  }},
{multi: true} )
```

MongoDB – Modificar

- Reemplazar un documento entero: basta con pasar un documento como segundo argumento y sólo mantendrá el id
  - db.personas.update({Nombre: "Luis"}, {Apellidos: "García Campo", Edad: 19, Direccion:{ Calle:"Mi calle", Numero:5} })
- Upsert() si ningún documento cumple la condición de búsqueda crea uno nuevo
  - db.personas.update(

```
{ Nombre: "Lola" },
{
  $set: {
    Apellidos: "cambiado",
    Aficiones: ["triatlon", "Nadar" ]
  }},
{ upsert: true}
)
```

MongoDB – Borrar

- Borrar todos los documentos de una colección
  - db.personas.remove({})
- Borrar todos los documentos de una colección que cumplen una condición
  - Expresamos las condiciones como ya hemos visto
  - db.personas.remove({Nombre: "Lola"})
- Borrar un único documento de una colección que cumple una condición
  - db.personas.remove({Nombre: "Luis"},1)

Actividad

- Ejecuta las sentencias de las transparencias anteriores de modificación y borrado (excepto la que borra todos los documentos)
- Asegúrate de entenderlas

Actividad

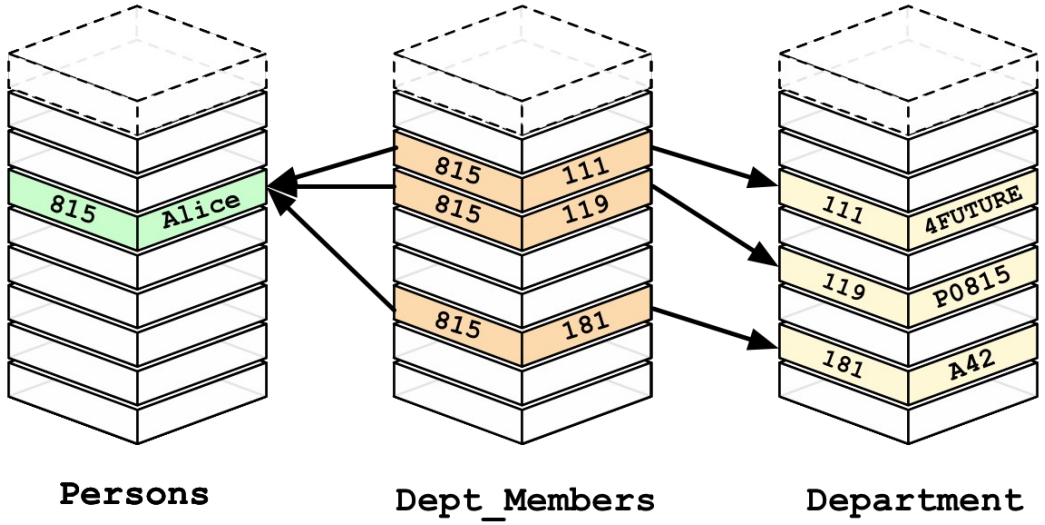
- Realizar Ejercicios1.pdf

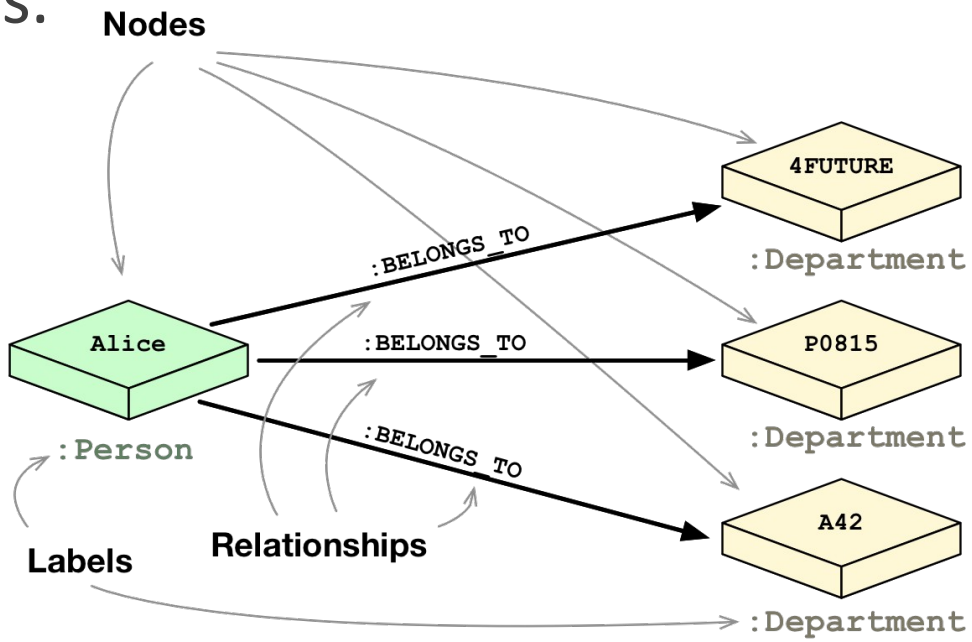
Neo4J

- **Neo4J** es un sistema de base de datos **multiplataforma** de **código abierto** orientado a **grafos**, altamente escalable, de **esquema libre** y escrito en Java.
- **¿Cuándo utilizarlo?**
  - Cualquier aplicación que necesite almacenar datos altamente escalables y altamente conexionados.
    - Múltiples relaciones entre datos que en un Sistema Relacional implicarían claves foráneas, tablas asociación y múltiples JOIN para relacionarlos (degradación del rendimiento alta)
  - Soporte para transacciones ACID
  - Escalabilidad a millones de nodos y relaciones (Distribuida)
  - Consultas eficientes para relacionar los datos mediante “traversals” (~ JOIN)

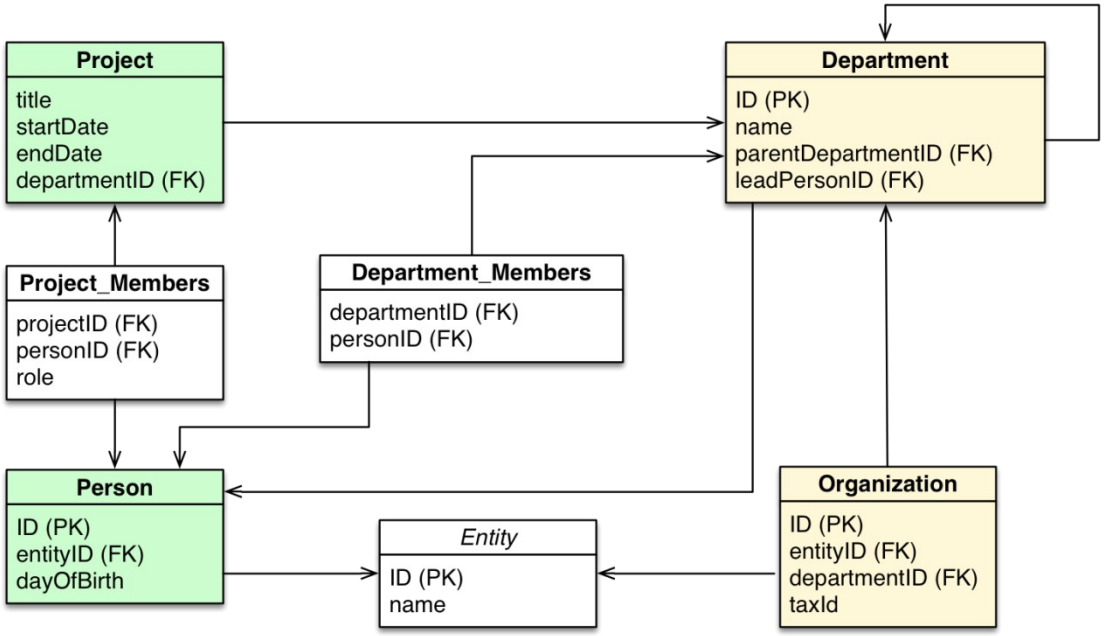
Neo4J

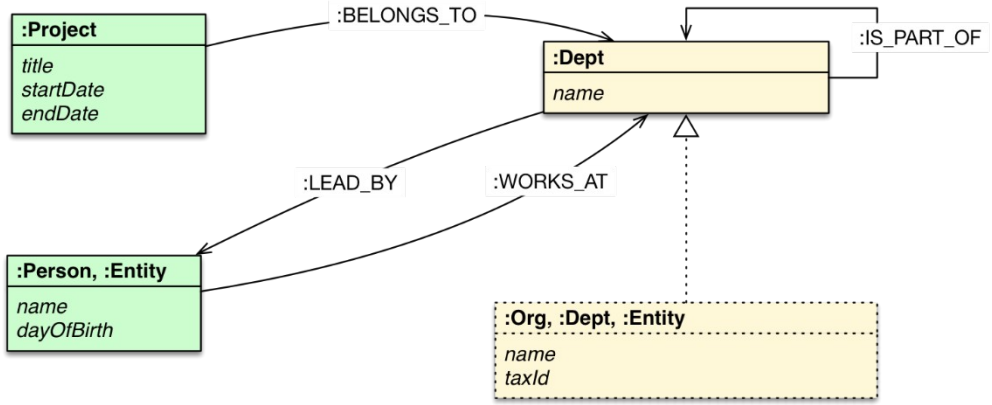
- **¿Cuándo NO utilizarlo?**
- Si los datos a manejar no están altamente conexionados (muchas relaciones) y no son altamente escalables.





Neo4J


Project
title
startDate
endDate
departmentID (FK)
Department
ID (PK)
name
parentDepartmentID (FK)
leadPersonID (FK)
Project_Members
projectID (FK)
personID (FK)
role
Department_Members
departmentID (FK)
personID (FK)
Person
ID (PK)
entityID (FK)
dayOfBirth
Entity
ID (PK)
name
Organization
ID (PK)
entityID (FK)
departmentID (FK)
taxId


:Project
title
startDate
endDate
:BELONGS_TO
:Dept
name
:IS_PART_OF
:LEAD_BY
:WORKS_AT
:Person, :Entity
name
dayOfBirth
:Org, :Dept, :Entity
name
taxId

Neo4J

- **Instalación**
  - Descargamos el .exe de la versión Community en http://neo4j.com/download/, si nos hace falta otra versión según nuestro SO podemos descargarlo de http://neo4j.com/download/other-releases/
  - En nuestro caso, en un OS Windows, ejecutamos el .exe descargado
  - Se abre un wizard en el que vamos pulsando en Siguiente
  - Al finalizar tenemos instalado en **C:\Program Files (x86)\Neo4j Community**:
    - Server – Neo4JServer
    - Client – Accesible via web

Neo4J

- **Instalación**
  - Lo primero que tenemos que hacer es arranvar el servidor de BD
  - Para ello accedemos a Todos los programas → "Neo4j Community" → "Neo4j Community“
  - Elegimos una ubicación para los archivos de BD y pulsamos en **Start**
  - Ahora que tenemos arrancado el servidor de BD para accede al cliente en un navegador web escribimos http://localhost:7474/browser/
  - Cuando fianalizemos pulsamos en **Stop** para parar el servidor de BD

Actividad

- **Instalación**
  - Instala Neo4J en tu equipo
  - Arranca al servidor
  - Accede al cliente vía web
  - Conéctate con el usuario y contraseña por defecto neo4j/neo4j
  - Te solicitará que cambies la contraseña, asegúrate de recordarla.

Neo4J

- **Conceptos**
- En Neo4J tenemos grafos, nodos, propiedades con valores, etiquetas, relaciones y operación “Traversal”

RDBMS	Graph Database
Tables	Graphs
Rows	Nodes
Columns and Data	Properties and its values
Constraints	Relationships
Joins	Traversal

Neo4J - Nodos

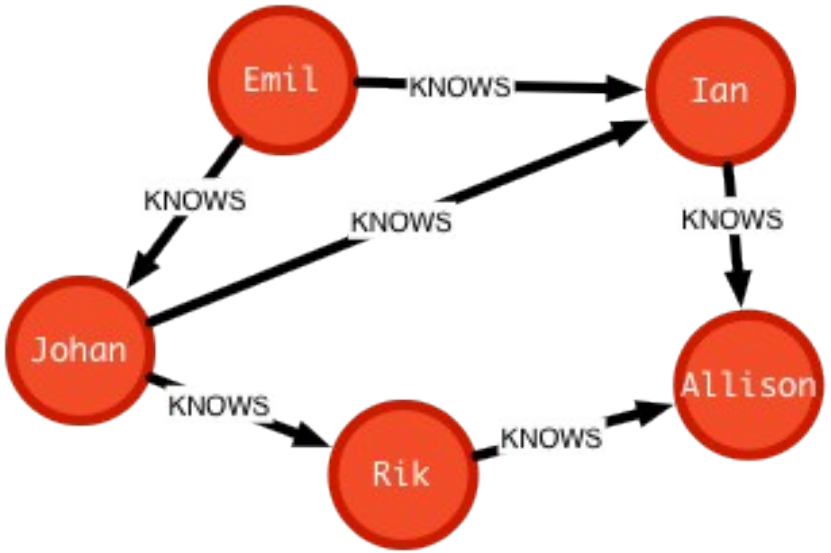
- **Grafo:** un grafo es un conjunto de nodos y relaciones que conectan esos nodos. Los nodos y las relaciones tienen propiedades que representan los datos. Una propiedad es un par clave-valor.
- **Nodo**: un nodo es el registro de datos en un grafo. Se representan mediante un círculo. Los datos se almacenan como propiedades (pares clave-valor) que pueden ser cadenas, números o booleanos http://neo4j.com/docs/stable/property-values-detailed.html
  - Nodos similares pueden tener propiedades diferentes

Neo4J - Etiqueta

- **Etiqueta:** nos permite agrupar nodos. Por ejemplo, la etiqueta Persona para cada nodo que representa a una persona.
  - Un nodo puede tener 0 o más etiquetas
  - Las etiquetas no tienen propiedades
  - Una etiqueta la representamos con un color

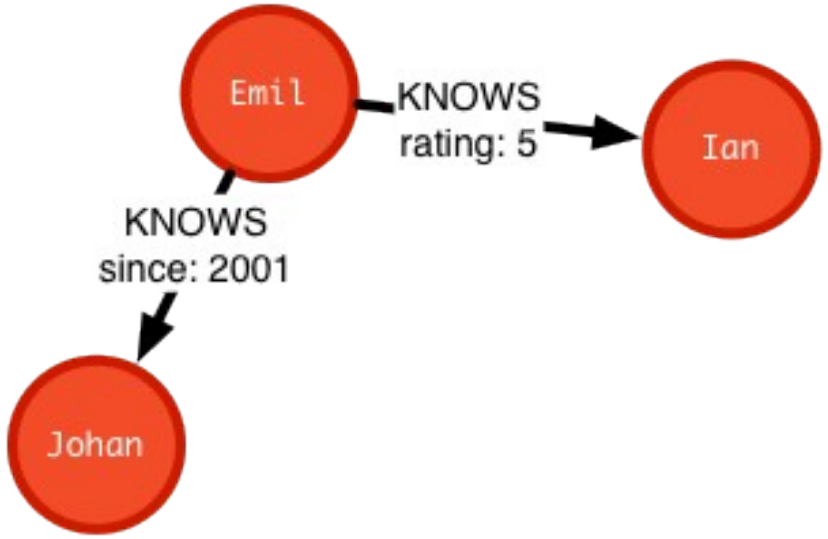
Neo4J - Relaciones

- **Relación:** nos permite conectar nodos.
  - Siempre son direccionales (unidireccionales o bidireccionales), cada una tiene uno nodo de inicio y un nodo de fin
  - Siempre tienen un tipo
  - Se representan como una flecha dirigida



Neo4J - Relaciones

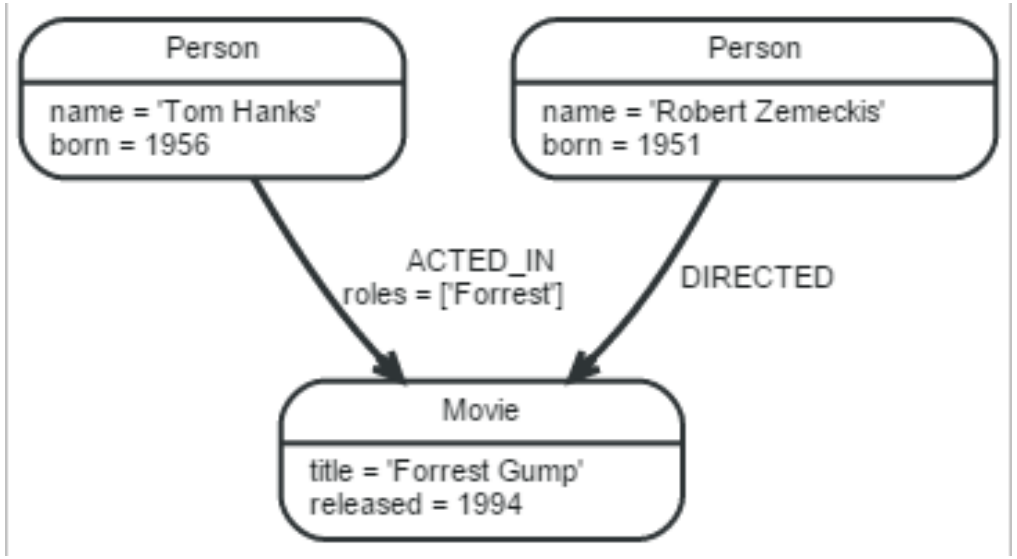
- **Relación:**
  - Las relaciones también pueden tener propiedades



Neo4J - CQL

- **CQL: Cypher Query Language**
  - Es el lenguaje que utiliza Neo4J para crear, consultar, modificar y borrar los grafos.
  - Vamos a ir viendo CQL sobre un Ejemplo Concreto de Grafo

CQL Comando	USO
CREATE	Crear nodos, relaciones y propiedades
MATCH	Recuperar datos de nodos, relaciones y propiedades
RETURN	Devolver resultados de consultas
WHERE	Condiciones para filtrar los datos recuperados
DELETE	Borrar nodos y relaciones
REMOVE	Borrar propiedades de nodos y relaciones
ORDER BY	Ordenar datos recuperados
SET	Añadir o actualizar etiquetas



Neo4J - CQL

- **Creación de Nodos**

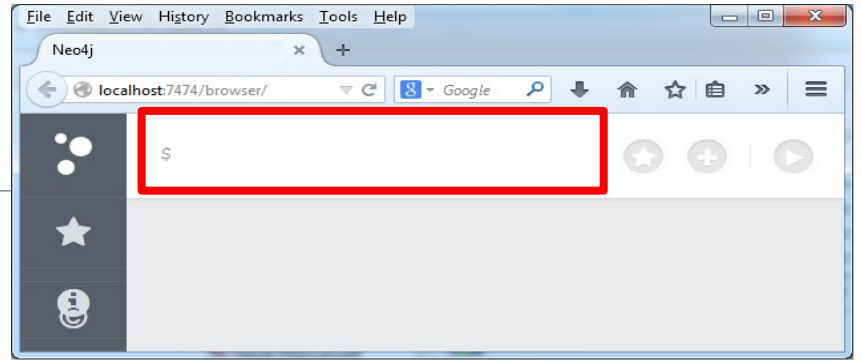
```
CREATE (
  <node-name>:<label-name>
  {
    <Property1-name>:<Property1-Value>
    <Propertyn-name>:<Propertyn-Value>
  }
)
```

```
CREATE (t:Person {name:‘Tom Hanks', born:1956})
RETURN t
```

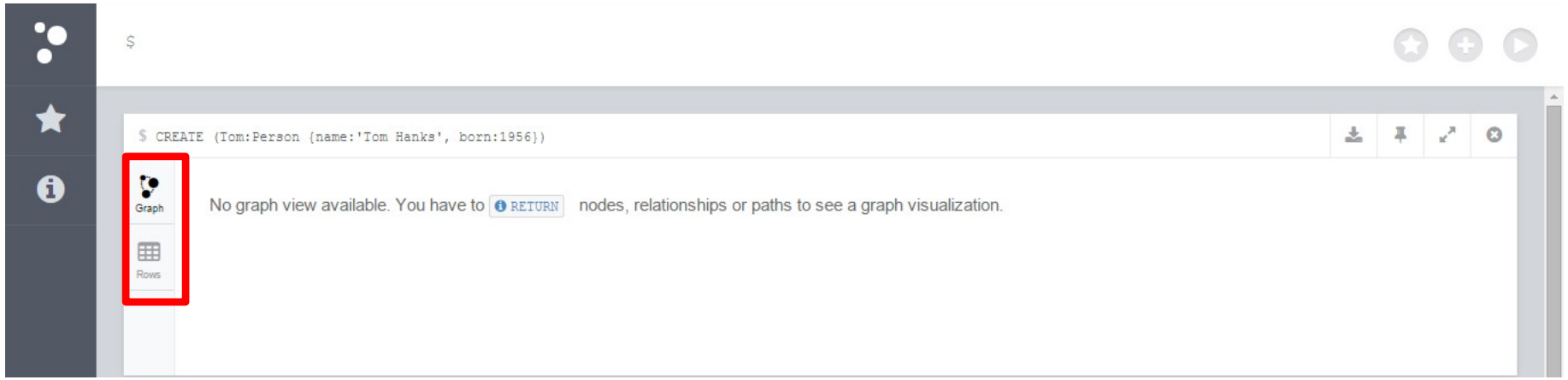
El nombre del nodo “es como una variable” se utiliza para “alamecenar nodos” al crear y consultar pero no es una referencia única a cada nodo que podamos utilizar para consultar. No existe uno nodo t, Internamente a cada nodo se le asigna un identificador númerico,

Neo4J crea de forma automática una propiedades ID numérica autoincremental para identificar a nodos y relaciones.

Actividad

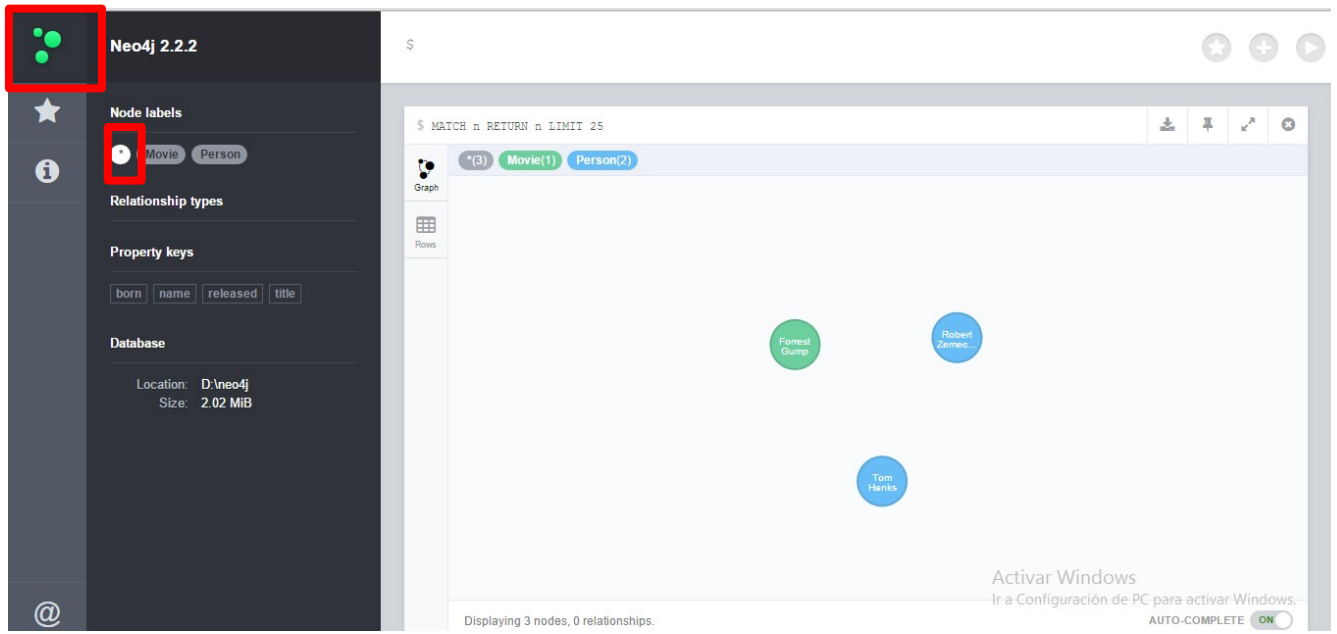


- **Ejecuta la sentencia anterior CREATE en el cliente web de Neo4J**
- Escríbela en el prompt y pulsa en el play
- Una vez creado el nodo aparece una ventana nueva en la que luego podremos ver el grafo creado o los registros de datos. Explora ambas vistas y familiarízate
- Escribe y ejecuta las sentencias create necesarias para crear los otros 2 nodos de nuestro ejemplo, después explora las vistas de grafo y registros



Actividad

- Para ver ahora todos los nodos de nuestro grafo pincha en el siguiente icono que aparece al desplegar la barra de navegación izquierda
  - Prueba ahora las vistas de grafo y registros
  - Fijáte en la propiedad Id
  - Prueba a seleccionar la etiqueta Movie sólo
  - Prueba a seleccionar la etiqueta Person sólo



Neo4J - CQL

- **Consulta de Información**
- Con Match podemos consultar información sobre los nodos y propiedades y sobre relaciones y propiedades
- MATCH

```
(
  <node-name>:<label-name>
) RETURN
  <node-name>.<property1-name>,
  ...
  <node-name>.<propertyn-name>
```

No se puede usar de forma aislada, se tiene que usar junto con RETURN para recuperar información o junto con update para actualizarla

Consultamos por los nodos etiquetados como persona

```
MATCH (t:Person)
RETURN t
```

```
MATCH (t:Person)
RETURN t.born
```

Neo4J - CQL

- **Consulta con condiciones WHERE**
- WHERE <condition> <boolean-operator> <condition>
- <condition> : <property-name> <comparison-operator> <value>
- Operadores Booleanos: AND, OR, XOR, NOT
- Operadores de comparación: =,<>,<,>,<=,>=

```
MATCH (t:Person)
WHERE t.name='Tom'
RETURN t.born
```

Actividad

- Ejecuta las consultas anteriores
- Fíjate que cuando devolvemos propiedades en vez del nodo no tenemos acceso a la vista de grafo
- Escribe consultas para obtener
  - las propiedades name y born de los nodos Person
  - la propiedade title de los nodos Movie
- Ahora intenta hacer las consultas anteriores con 2 MATCH y un único RETURN
- Escribe una consulta para obtener el titulo de las películas estrenadas en 1994

Neo4J - CQL

- **Relaciones**

- Con la sentencia CREATE podemos crear relaciones direccionales que pueden ser unidireccionales o bidireccionales

- Además podemos crear relaciones con propiedades
  - **Relación unidireccional sin propiedades sobre nodos existentes**

```
MATCH (<node1-name>:<node1-label-name>),(<node2-name>:<node2-label-name>)
CREATE
        (<node1-name>)-[<relationship-label-name>:<relationship-name>]->(<node2-name>)
RETURN <relationship-label-name>
```

```
MATCH (a:Person),(b:Movie)
WHERE a.name = ‘Robert Zemeckis' AND b.title = 'Forrest Gump'
CREATE (a)-[r:DIRECTED]->(b)
RETURN r
```

Neo4J - CQL

- **Relaciones**
  - **Relación unidireccional con propiedades sobre nodos existentes**

```
MATCH (<node1-label-name>:<node1-name>),(<node2-label-name>:<node2-name>)
CREATE
        (<node1-label-name>)-[<relationship-label-name>:<relationship-name>
        {<define-properties-list>}]->(<node2-label-name>)
RETURN <relationship-label-name>
```

```
MATCH (a:Person),(b:Movie)
WHERE a.name = ‘Tom Hanks' AND b.title = 'Forrest Gump'
CREATE (a)-[r:ACTED_IN{Roles:’Forrest’}]->(b)
RETURN r
```

Neo4J - CQL

- **Relación unidireccional con propiedades y creación de nodos**

```
CREATE
        (<node1-label-name>:<node1-name>{<define-properties-list>})-
        [<relationship-label-name>:<relationship-name>{<define-properties-list>}]
        ->(<node1-label-name>:<node1-name>{<define-properties-list>})
RETURN <relationship-label-name>
```

```
CREATE (keanu:Actor { name:'Keanu Reeves' })-[rel:ACTED_IN { role : 'Neo' }]->(matrix1:Movie { title :
'The Matrix', year : '1999-03-31' })
RETURN keanu,rel,matrix1
```

Neo4J - CQL

- **Consultar información de relaciones**

MATCH

(<node1-label-name>)-[<relationship-label-name>:<relationship-name>]->(<node2-label-name>)

RETURN <relationship-label-name>

```
MATCH (a)-[r:ACTED_IN]->(b)
RETURN a,r,b
```

Actividad

- Ejecuta los ejemplos de creación y consulta de relaciones y asegúrate de entenderlos
- Después prueba la siguiente sentencia, ¿Qué hace?
- CREATE p =(andres:Person { name:'Andres' })-[:WORKS_AT]->(neo:Teatro{name: 'neo'})<-[:WORKS_AT]-(Michael:Person { name:'Michael' })

  RETURN p
- Escribe sentencias para consultar los nodos que mantengan una relación WORKS_AT
- Explora tanto el grafo como los registros de datos

Neo4J - CQL

- **Borrar Nodos**
  - DELETE <node-name-list>
- **Borrar Nodos y Relaciones**
  - DELETE <node1-name>,<node2-name>,<relationship-name>
- **Añadir propiedades a Nodos y Relaciones**
  - SET <property-name-list>
- **Borrar propiedades y etiquetas**
  - REMOVE <property-name-list>
    - Operador coma para separar listas de etiquetas
    - Operador punto para separar propiedades de nodos

```
MATCH (a:Person) Delete a

MATCH (p:Person)-[r]-(l:Teatro) DELETE p,r,l
```

```
CREATE (book:Book:Paper {id:122,title:"Neo4j
Tutorial",pages:340,price:250})
RETURN book

MATCH (book { id:122 })
SET book.editorial = "IES"
RETURN book

MATCH (book { id:122 })
REMOVE book.price
RETURN book

MATCH (b:Book)
REMOVE b:Paper
RETURN b
```

Actividad

- Ejecuta los ejemplos de sentencias de la transparencia anterior
- Asegúrate de entender lo que hace cada una de ellas

Neo4J –Operadores y Funciones

- ORDER BY <property-name-list> [DESC]
- IN[<Collection-of-values>]
- LIMIT <number>
- UPPER,LOWER, SUBSTRING, REPLACE
- COUNT, MAX, MIN, SUM, AVG

```
MATCH (emp:Employee)
RETURN emp
LIMIT 2

MATCH (emp:Employee)
RETURN emp.id,emp.name,emp.sal,emp.deptno
ORDER BY emp.name

MATCH (e:Employee)
WHERE e.id IN [123,124]
RETURN e.id,e.name,e.sal,e.deptno

MATCH (e:Employee)
RETURN e.id,UPPER(e.name),e.sal,e.deptno

MATCH (e:Employee)
RETURN MAX(e.sal),MIN(e.sal)
```

Neo4J –Funciones sobre relaciones

- STARTNODE: devuelve el nodo inicial de una relación
- ENDNODE: devuelve el nodo final de una relación
- ID: devuelve el identificador de una relación.
- TYPE: devuelve la representación en cadena del tipo de una relación

```
MATCH (a)-[r:WORKS_AT]->(b)
RETURN STARTNODE(r)

MATCH (a)-[r:WORKS_AT]->(b)
RETURN ENDNODE(r)

MATCH (a)-[r:WORKS_AT]->(b)
RETURN ID(r),TYPE(r)
```

Actividad

- Crea 5 nodos con etiqueta ‘Employee’ siguiendo esta estructura:
  - CREATE (emp:Employee{id:123,name:"Lokesh",sal:35000,deptno:10})
  - Utiliza los id 122,123,124,125,126
- Después ejecuta los ejemplos de sentencias de las transparencias anteriores
- Explora las vistas de grafo y datos
- Asegúrate de entender lo que hace cada una de ellas

Actividad

- Para practicar Ejercicios2.pdf

Referencias

1. WebBlogs, http://www.genbetadev.com/bases-de-datos/bases-de-datos-nosql-elige-la-opcion-que-mejor-se-adapte-a-tus-necesidades
2. MongoDB, http://www.mongodb.org/
3. MongoDB tutorial, http://www.genbetadev.com/bases-de-datos/mongodb-que-es-como-funciona-y-cuando-podemos-usarlo-o-no
4. MongoDB tutorial, http://www.genbetadev.com/bases-de-datos/una-introduccion-a-mongodb
5. Neo4J, http://neo4j.com/
6. Tutorial Neo4J, http://www.tutorialspoint.com/neo4j/